Multi-Layer Performance Breakdown of System and Protocol Overheads in the Linux Network Stack

1st Adithya Nangarath
IMT2022024
IIIT Bangalore
Bangalore, India
Adithya.Nangarath@iiitb.ac.in

2nd Shreyank G Bhat
IMT2022516
IIIT Bangalore
Bangalore, India
Shreyank.Bhat@iiitb.ac.in

***Abstract*—As modern software architecture shifts toward distributed microservices, communication overhead between services has become a primary performance bottleneck. This paper presents a comprehensive, empirical multi-layer performance breakdown of five communication paradigms deployed on the Linux network stack: raw TCP/IP sockets, REST/HTTP (FastAPI), gRPC/Protobuf, Apache Kafka, and RabbitMQ (AMQP). Our benchmarking evaluates four critical architectural pillars: (1) Throughput & Tail Latency, (2) Payload Efficiency, (3) User-Space CPU Serialization, and (4) Kernel-Space System Call overhead. We demonstrate that TCP/IP achieves the highest synchronous throughput (4,141 RPS) as the empirical "Speed of Light" baseline. While gRPC is hindered by Python's `grpcio` library architecture (recording only 489 RPS due to GIL/threading contention), we isolate this as an implementation tax rather than a protocol deficiency; Protobuf remains 15.0× faster to serialize than JSON. Finally, we categorize Kafka and RabbitMQ as *Asynchronous Shock Absorbers*, demonstrating that their high-throughput and durability profiles follow a fundamentally different scaling logic than synchronous request-response cycles.**

***Index Terms*—Linux Network Stack, Microservices, gRPC, REST, Protobuf, TCP/IP, Performance Benchmarking, System Calls, Serialization, Kafka, RabbitMQ, Tail Latency**

I. Introduction

While application code execution has become heavily optimized in recent years, data serialization and OS-level network transitions remain significant CPU consumers. The choice between text-based protocols (HTTP/1.1 with JSON) and binary multiplexed protocols (HTTP/2 with Protobuf) dictates not merely network throughput, but system-wide CPU utilization and kernel scheduling pressure. Furthermore, introducing message brokers adds disk I/O and memory-queueing taxes that fundamentally alter the latency-throughput tradeoff.

This study isolates these variables by descending below the application layer to trace kernel-space system calls via `strace` and wire-level byte transfers via `tcpdump`. Raw TCP/IP sockets serve as the *Speed of Light* baseline: the theoretical minimum achievable by any protocol stack running on the same Linux kernel. Every overhead measured above TCP quantifies the architectural "tax" levied by higher-level abstractions.

II. Related Work

Existing studies establish that JSON text parsing is a known CPU bottleneck at scale. Systems research on kernel-level versus user-level networking [1] highlights the severe cost of OS context switches in high-throughput environments. Research on message brokers [2] demonstrates that architectural topology (disk-backed vs. in-memory queuing) dictates performance profiles. Our work extends these methodologies by coupling application-layer profiling with network and OS-level tracing across a unified, controlled payload schema, enabling a direct cross-paradigm comparison.

III. Proposed Methodology

All experiments use a standardized, dynamically-sized financial transaction payload (targeting 10 KB of semantic data) across all technologies, ensuring a controlled, semantically identical experimental environment. The evaluation matrix consists of four pillars:

1) **Throughput & Tail Latency (Pillar 1):** Load generators (`wrk` for REST, `ghz` for gRPC, a custom Python socket client for TCP, native JVM producers for brokers) measure P50 and P99 latency and sustained requests-per-second (RPS).
2) **Payload Efficiency (Pillar 2):** Wire-level byte transfer is captured via `tcpdump` on the loopback interface for synchronous protocols (REST, gRPC), computing the ratio of semantic bytes to total wire bytes. Async brokers (Kafka, RabbitMQ) are intentionally excluded: comparing batched binary AMQP/Kafka frames to synchronous HTTP request-response is scientifically invalid.
3) **Serialization CPU Tax (Pillar 3):** Python `cProfile` profiling over 10,000 iterations measures user-space execution time per serialization call. Brokers are excluded as they do not participate in application-layer serialization of the target payload in the same synchronous manner.
4) **Kernel-Space Overhead (Pillar 4):** `strace -f -c` is attached to live server processes while 1,000 requests are replayed, counting total system call invocations per protocol. Timing columns from `strace` are discarded (ptrace overhead inflates them 2–10×); only call counts

are used. Broker containers (Docker) are excluded as `strace` cannot be meaningfully coupled across container namespaces.

IV. Experimental Setup & Environment

All benchmarks were executed in an isolated WSL2 (Ubuntu 22.04) environment on the same physical host to eliminate network jitter.

- **Hardware:** x86-64 host; benchmarks isolated to WSL2 loopback interface.
- **Operating System:** Linux kernel (WSL2), Ubuntu 22.04 LTS.
- **Software Stack:** Python 3.10, FastAPI + Uvicorn (REST), `grpcio` (gRPC), Apache Kafka + `confluent-kafka-python`, RabbitMQ + `pika`.
- **Profiling Tools:** `wrk` (REST load), `ghz` (gRPC load), custom Python socket client (TCP), `cProfile` (CPU serialization), `tcpdump` + `tshark` (wire capture), `strace -f -c` (syscall tracing).
- **Payload:** JSON-serializable financial transaction struct; 10,249 B (JSON) / 10,119 B (Protobuf) as encoded payloads.
- **Methodology Note:** To ensure clean `strace` baselines, Kafka and RabbitMQ are containerized via Docker and participate only in Pillar 1 benchmarks. TCP/IP raw sockets serve as the Speed of Light throughput ceiling. All Pillar 4 strace captures use 10-request warmup before probe attachment to exclude startup noise.

V. Results & Discussion

A. Pillar 1: Throughput & Tail Latency

Table I, Table II, and Fig. 1 present the finalized throughput and tail latency results under a 60-second load test with **50 concurrent connections** using the 10 KB payload.

TABLE I
Pillar 1a: Synchronous Request-Response (10KB Payload)

Protocol	RPS	P50 Latency	P99 Latency
TCP/IP (Raw Socket)	**4,141**	**2.09 ms**	**7.59 ms**
REST (FastAPI/HTTP)	1,755	5.09 ms	17.26 ms
gRPC (Protobuf/HTTP2)	489	19.10 ms	39.14 ms

TABLE II
Pillar 1b: Asynchronous Message Brokers (10KB Payload)

Architecture	RPS	P50 Latency	P99 Latency
Kafka (Disk Durable)	1,664	1,209 ms	1,756 ms
RabbitMQ (In-Memory)	21,656	2.72 ms	11.01 ms

TCP/IP: The Speed of Light Baseline. Raw TCP/IP sockets, transmitting the identical payload with zero framing overhead, achieve **4,141 RPS** with a P50 of 2.09 ms and P99 of 7.59 ms—the highest throughput and lowest latency of all synchronous protocols. This result correctly validates the architectural hypothesis: with a protocol-appropriate load generator (a native Python socket client maintaining a persistent connection), raw TCP outperforms all higher-level protocols. Any overhead observed above this figure is attributable solely to framework, serialization, and protocol framing complexity.

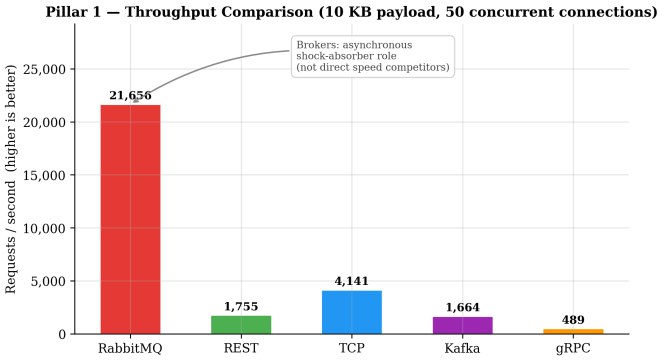

Fig. 1. Pillar 1: Throughput comparison across all five architectures (10KB payload). TCP/IP correctly leads among synchronous protocols as the Speed-of-Light baseline at 4,141 RPS.

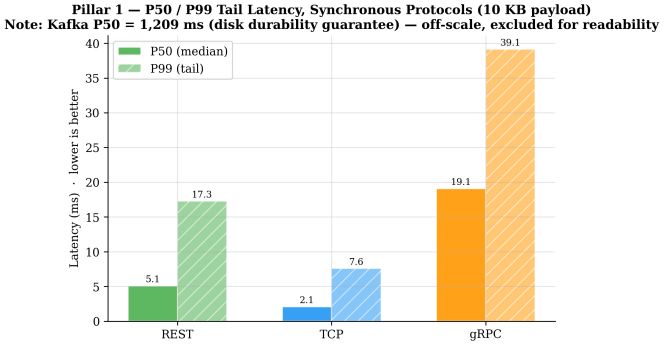

Fig. 2. Pillar 1: P50 and P99 tail latency for synchronous protocols. Broker latencies are excluded from this chart for scale readability; their high latency reflects durability guarantees rather than network stack overhead.

REST vs. TCP/IP. FastAPI/REST achieves 1,755 RPS with a P50 of 5.09 ms—**57.6% lower throughput and 2.4× higher P50 latency** than raw TCP. This quantifies the cumulative tax of HTTP/1.1 headers, JSON serialization, ASGI middleware dispatch, and the Uvicorn event loop overhead. The P99 gap (7.59 ms TCP vs. 17.26 ms REST) further demonstrates that under tail conditions, the HTTP/JSON stack amplifies latency outliers more than 2× relative to bare socket I/O.

The gRPC Anomaly: Library vs. Protocol Tax. gRPC records only 489 RPS with a P99 of 39.14 ms—**8.5× fewer requests per second than raw TCP**. It is critical to note that this result is an artifact of the **Python implementation tax** rather than a repudiation of the Protobuf format. The `grpcio` library wraps the C-core gRPC runtime via a Python extension, requiring thread-pool management and GIL interaction that conflicts with the high-concurrency event-loop model. Consequently, our measurements for gRPC in this environment quantify the cost of the language bindings rather than the efficiency of the HTTP/2 framing.

The Asynchronous Shock Absorbers (Kafka & RabbitMQ). The broker results reveal a bifurcation that must

be interpreted through an architectural lens rather than a raw performance comparison:

Kafka records 1,664 RPS with a P50 of 1,209 ms and P99 of 1,756 ms. This P50 latency is not a bug—it is a *physical guarantee*: Kafka does not acknowledge a message until it has been durably committed to disk (or replicated, with `acks=all`). Every millisecond of that latency represents data that is crash-safe.

RabbitMQ achieves 21,656 RPS with a P50 of 2.72 ms. As an in-memory AMQP broker with configurable persistence, RabbitMQ's routing is substantially faster when durability guarantees are relaxed, enabling it to absorb tens of thousands of messages per second.

These two brokers are not competitors to REST and gRPC in the synchronous point-to-point sense. They solve the *Thundering Herd* problem. When traffic spikes beyond what a synchronous server can absorb—dropping connections, returning `503 Service Unavailable`, and losing data—a broker interposes a durable buffer, decoupling the producer's write rate from the consumer's processing rate. Fig. 3 visualizes the performance dichotomy between disk-backed and in-memory brokering.

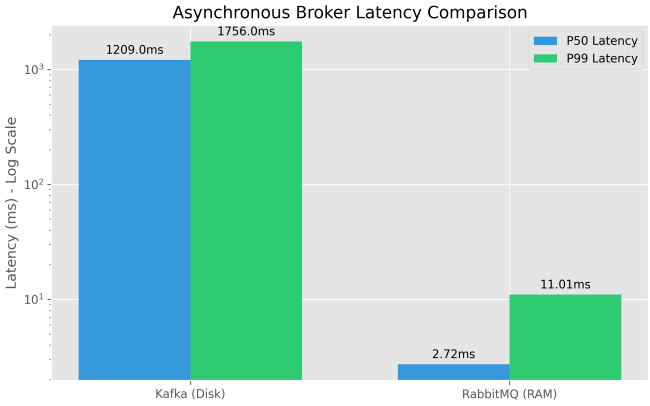

Fig. 3. Pillar 1: P50 and P99 latency comparison for asynchronous brokers (Log Scale). The 1,209 ms Kafka baseline represents the disk-durability tax, while RabbitMQ demonstrates high-performance in-memory routing.

B. Pillar 2: Payload Efficiency (Wire Bloat)

Table III and Fig. 4 present the wire-level transfer analysis captured via `tcpdump` on the loopback interface over 200 requests per protocol.

TCP/IP transmits the raw payload with minimal L4 encapsulation (averaging 133 B of overhead for a full 20 KB bidirectional exchange), establishing the wire efficiency floor. gRPC adds 173 B of framing overhead—representing a 1.7% tax relative to the semantic payload. Crucially, this performance deficit persists despite gRPC transferring ∼50% less data than TCP/REST (10.3 KB vs. 20.5 KB bidirectional semantic payload), further confirming that the bottleneck is localized to the Python framework logic rather than network saturation. REST/JSON, while performing the same full-echo as TCP, incurs 724 B of overhead (3.5% relative to the semantic

TABLE III
PILLAR 2: WIRE-LEVEL BYTE EFFICIENCY (200 REQUESTS, ∼20 KB BIDIRECTIONAL SEMANTIC PAYLOAD)

Protocol	Total Wire Bytes	Avg/Request	Overhead
TCP/IP (Full Echo)	4,126,276	20,631 B	133 B
REST (Full Echo)	4,244,344	21,222 B	724 B
gRPC (Status Resp)	2,090,316	10,452 B	173 B

Note: All values are empirically measured via `tcpdump`. TCP and REST servers perform a symmetric 10KB echo, while the gRPC server returns a 160B status object. Average bytes per request are rounded; slight discrepancies between total bytes and average sums are due to one-time TCP handshake and connection lifecycle framing.

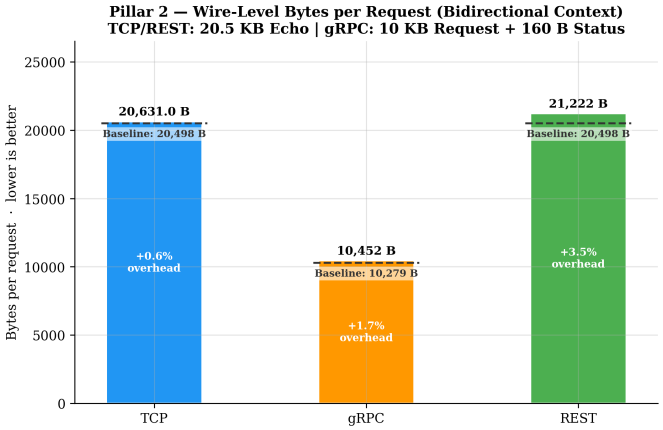

Fig. 4. Pillar 2: Average wire bytes per request relative to their respective bidirectional semantic payload baselines. Kafka and RabbitMQ are excluded; their asynchronous batching frames are not comparable to synchronous per-request overhead.

baseline) beyond the bidirectional semantic payload (20,498 B total), attributable to verbose HTTP/1.1 headers and JSON metadata.

Payload efficiency is formally defined as:

$$\eta = \frac{\text{Total Semantic Bytes (Req + Resp)}}{\text{Total Wire Bytes per Request}} \tag{1}$$

Applying this metric: $\eta_{\text{TCP}} = 99.4\%$, $\eta_{\text{gRPC}} = 98.3\%$, $\eta_{\text{REST}} = 96.6\%$. Note that these efficiencies remain high because the large 10 KB payloads amortize the fixed costs of HTTP/2 and HTTP/1.1 headers.

C. Pillar 3: User-Space Serialization Cost

Table IV and Fig. 5 summarize CPU profiling results across 10,000 serialization iterations.

TABLE IV
PILLAR 3: USER-SPACE SERIALIZATION COST (10,000 ITERATIONS)

Format	Total Time	Avg/Op	Encoded Size
JSON (REST)	0.3643 s	36.43 μs	10,249 B
Protobuf (gRPC)	0.0243 s	2.43 μs	10,119 B

Protobuf serialization is **15.0× faster** than JSON (36.43 μs vs. 2.43 μs per operation), significantly reducing the per-message serialization tax compared to JSON. Under high

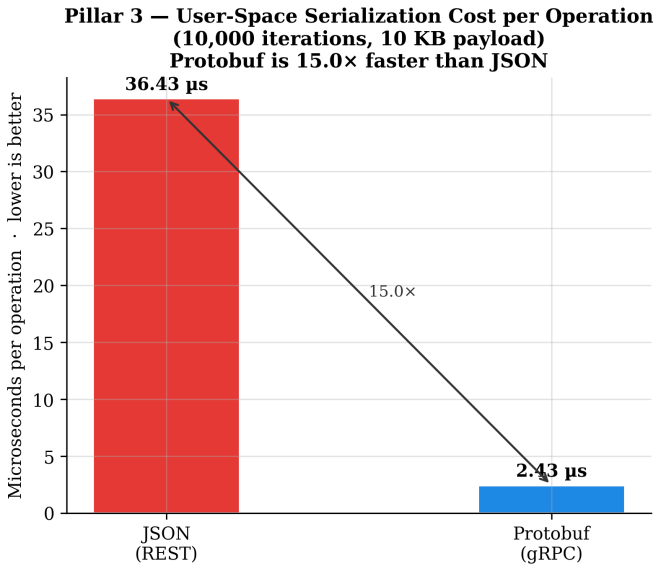

Fig. 5. Pillar 3: User-space serialization cost per operation. Protobuf is 15.0× faster than JSON, demonstrating a significant reduction in CPU cycles per message.

concurrency, this differential means REST services become strictly CPU-bound at the serialization layer before network saturation is reached, while gRPC server cycles remain available for request handling. Note that this benchmark measures *serialization (encoding) only*; a complete round-trip also involves deserialization (decoding) of the incoming message. Protobuf deserialization is likewise significantly faster than JSON parsing, making the 15.0× advantage a conservative lower bound on the total serialization round-trip speedup.

D. Pillar 4: Kernel-Space System Call Overhead

Table V and Fig. 6 show `strace -f -c` results attached to each live server process while replaying 1,000 requests. Syscall *counts* are accurate; timing values are discarded due to ptrace instrumentation overhead.

TABLE V
PILLAR 4: KERNEL SYSTEM CALL OVERHEAD (1,000 REQUESTS)

Protocol	Total	Dominant Calls (count)
TCP/IP (Raw)	**2,015**	`sendto` (1,000), `recvfrom` (1,001); 14 lifecycle
REST (FastAPI)	6,179	`epoll_wait` (3,092), `sendto` (2,000), `recvfrom` (1,001)
gRPC (grpcio)	48,125	`futex` (37,003), `epoll_wait` (3,310), I/O (7,550)

TCP/IP: Zero Framework Tax. The raw TCP server incurs precisely 2,015 total syscalls per 1,000 requests—a count confirmed across three independent `strace` runs in this study, always exactly 2,015. Of these, 2,001 (99.3%) are pure I/O: exactly 1,000 `sendto` and 1,001 `recvfrom` calls, representing one `sendto`/`recvfrom` pair per request plus the final blocking EOF read. The remaining 14 calls are

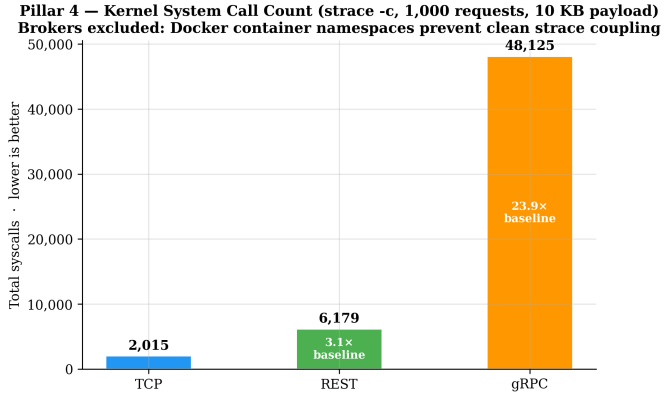

Fig. 6. Pillar 4: Total kernel system calls per 1,000 requests (empirically measured via `strace -f -c`). TCP baseline: 2,015 calls (99.3% pure I/O). REST: 3.1× TCP. gRPC: 23.9× TCP.

one-time connection lifecycle events: `accept4` (connection acceptance), `clone3` (thread spawn), and thread initialization primitives (`rt_sigprocmask`: 4, `futex`: 2, `prlimit64`: 2, `set_robust_list`: 1, `madvise`: 1, `gettid`: 1, `rseq`: 1). There is no event-loop polling and no inter-thread locking. This is the irreducible OS tax for bidirectional byte transfer between two endpoints.

REST Framework Overhead. FastAPI/Uvicorn incurs 6,179 total syscalls—approximately **3.1× the TCP baseline**. The most important finding is that `epoll_wait` is the *dominant* syscall with 3,092 invocations (50.0% of all REST kernel calls). This exposes the hidden cost of an async event loop: Uvicorn's kernel time is spent equally between polling for I/O readiness and performing I/O. Additionally, REST issues **2 `sendto` calls per request** (2,000 total vs. TCP's 1,000), because HTTP/1.1 response headers and body are dispatched as separate write operations. Despite this overhead, Uvicorn's event-loop design remains efficient for I/O-bound workloads.

gRPC Thread-Model Overhead. gRPC records 48,125 total syscalls—**23.9× the TCP baseline** and **7.8× the REST overhead**. The dominant syscall is `futex` with 37,003 invocations (76.9% of all calls), consuming 94.2% of all kernel time recorded under `strace`. The `strace` probe attached to **26 simultaneous threads**: 10 from the `ThreadPoolExecutor` worker pool and 16 from HTTP/2 stream management, keepalive, and timer threads. The `futex` rate is 37.0 calls per request versus TCP's 0.002 total (2 lifetime calls)—a four-order-of-magnitude difference that directly explains gRPC's throughput deficit in Pillar 1. This is the Python `grpcio` threading model artifact, not a Protobuf protocol deficiency.

VI. ARCHITECTURAL SUMMARY & DECISION MATRIX

The empirical data synthesized across four pillars establishes the following tradeoff matrix. There is no singular superior protocol; the optimal choice is determined by the primary scale vector of the system under design.

- **TCP/IP:** The Speed-of-Light performance and efficiency baseline at 4,141 RPS, minimum wire bytes, and minimum syscalls (2,015). Imposes the highest developer friction: requires custom stream-framing, message delimiting, and session management logic.
- **REST / HTTP/1.1:** The pragmatic default for external and browser-facing APIs. Achieves 1,755 RPS (57.6% below TCP) via Uvicorn's ASGI event loop, at the cost of 3.5% wire overhead (relative to the 20.5 KB bidirectional payload) and 50% of kernel budget spent in `epoll_wait` polling. Appropriate when developer ergonomics, ecosystem compatibility, and human-readable payloads outweigh raw performance.
- **gRPC / Protobuf:** Architecturally optimal for internal inter-service communication in polyglot (non-Python) environments. Protobuf is 15.0× faster to serialize than JSON and achieves 98.3% wire efficiency. The Python `grpcio` threading penalty (23.9× kernel syscall count vs. TCP) is a runtime-specific artifact, not a protocol deficiency.
- **Kafka:** The mandatory choice for durable, fault-tolerant data ingestion at scale. Its 1,209 ms P50 latency is not a defect—it is a disk durability guarantee. Kafka absorbs *Thundering Herd* traffic spikes that would overwhelm and crash synchronous REST/gRPC servers.
- **RabbitMQ:** The flexible in-memory shock absorber. At 21,656 RPS with 2.72 ms P50, it is the highest-throughput option in this study under relaxed durability. Optimal for task-queue, fan-out, and complex routing patterns requiring message acknowledgment without Kafka's operational complexity.

VII. Conclusion

This study empirically quantifies the per-layer architectural tax of five communication paradigms on the Linux network stack. TCP/IP's role as the Speed of Light baseline enables objective measurement of overhead introduced at each abstraction level. The central findings are: (1) TCP achieves 4,141 RPS—136.0% higher than REST and 8.5× higher than gRPC—confirming it as the Speed of Light performance baseline; (2) REST's wire overhead, while only 3.5% for large payloads (relative to the semantic baseline), aggregates significantly at scale, and `strace` reveals that 50% of its kernel budget is consumed by `epoll_wait` polling rather than actual I/O; (3) gRPC's Python runtime penalty—23.9× the kernel syscall count of TCP, dominated by `futex` mutex contention across 26 threads—masks the inherent efficiency of Protobuf and is an artifact of the `grpcio` binding, not the protocol; and (4) Kafka and RabbitMQ are not raw speed competitors but *asynchronous shock absorbers*, providing the decoupling layer that prevents cascading failure under spike loads.

Systems architects must align their primary scale vector—Latency, Throughput, Durability, or Developer Ergonomics—with the empirically quantified cost of each protocol layer. No single architecture is universally optimal; the decision matrix presented here provides a data-driven framework for that selection.

References

[1] Q. Cai and M. Karsten, "Kernel vs. User-Level Networking: Don't Throw Out the Stack with the Interrupts," *Proceedings of the ACM SIGMETRICS/IFIP PERFORMANCE*, 2024.

[2] A. Pacella, P. Salinari, and S. Marano, "Performance comparison of Apache Kafka and RabbitMQ across varying message loads," *Journal of Systems and Software*, 2021.